

Modeling and Control of Cyber-Physical Systems: Project Activity I Report

Coronelli Marco s348664
Givogre Riccardo s349452
Guidetti Matteo s348900

1 Task 1: Secure state estimation of a static CPS with sparse sensor attacks

1.1 Introduction

The goal of the first task is to implement two algorithms that both estimate the state of the CPS and which sensors are under attack, considering η to be a possible measurement noise. The implemented algorithms are: Inertial Jacobi Alternating Minimization (IJAM) and Iterative Soft Thresholding Algorithm (ISTA). We repeated the experiment multiple times testing different parameters; each time, averaging the results over 50 runs to get rid of statistical noise and obtain a clearer picture to work with.

1.2 Performance analysis

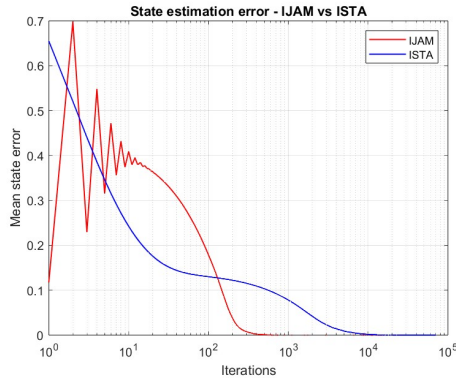


Figure 1: State estimation error

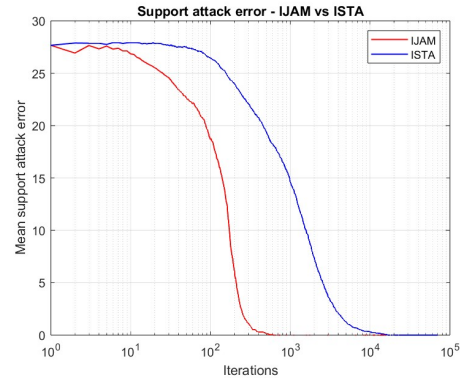


Figure 2: Support attack error

The differences between IJAM and ISTA are clearly illustrated in the two plots, which report the estimation errors for both the state and the attack support. Even if both algorithms eventually converge, their convergence rates differ significantly. After approximately 500 iterations: IJAM achieves a state estimation error of 10^{-2} , while ISTA still exhibits an error of magnitude 10^{-1} , reaching the same level of precision only after approximately 10,700 iterations. Regarding the support attack error, IJAM converges to 0 in about 700 iterations, meanwhile ISTA gives an error equal to 16, reaching 0 after around 14,800 iterations. These results indicate that, in this given setting, ISTA is roughly 20 times slower than IJAM in terms of convergence speed.

We can further analyse the two plots, dividing them into three distinct phases:

- **Phase I (iteration 0 to 20):** This initial stage displays some interesting patterns. Most notably, IJAM exhibits pronounced chattering in both the state and attack estimations, particularly visible in the state error. In contrast, ISTA follows a smoother, polynomial convergence trajectory. These differences reflect the intrinsic nature of the two algorithms: IJAM at each iteration attempts to solve the P-Lasso problem in one step, while ISTA performs a gradient descent, resulting in smaller updates between each iteration.
- **Phase II (iterations 20 to 100):** During this phase, the gap in support attack error between the two algorithms becomes more evident. However, ISTA appears to temporarily outperform IJAM in terms of state estimation accuracy, suggesting that ISTA's conservative updates may lead to more precise estimates in this range.
- **Phase III (beyond iteration 100):** This final phase clearly demonstrates the superior convergence properties of IJAM. It reaches accurate state and attack estimates nearly two orders of magnitude faster than ISTA.

In the following analysis, we investigate how different values of λ influence the performance of the algorithms. Here, λ is a design hyperparameter that controls the action of the soft-thresholding operator. Larger values of λ promote sparsity in the estimated attack vector by driving more entries toward zero. Conversely, smaller values of λ reduce sparsity, preserving more non-zero entries. However, for excessively large values of λ , the algorithms may produce an overly sparse attack vector, removing important non-zero components. This can reduce the accuracy of the estimation, as the algorithms focus more on fast convergence than on stability.

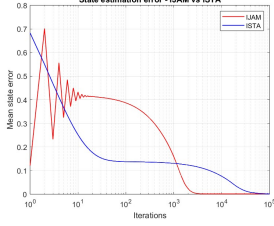


Figure 3: $\lambda = 0.01$

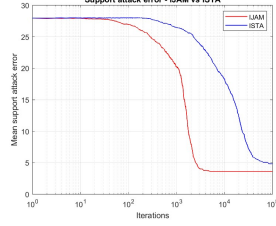


Figure 4: $\lambda = 0.4$

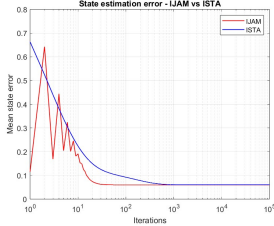
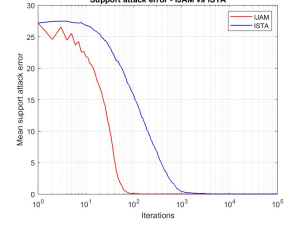
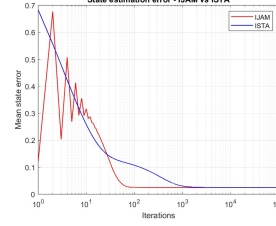


Figure 5: $\lambda = 1$

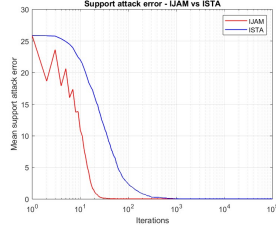
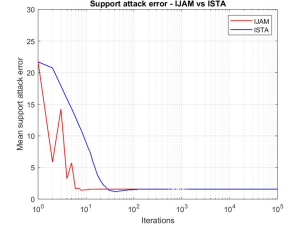
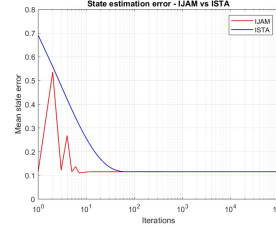


Figure 6: $\lambda = 3$



As expected, λ plays a crucial role in controlling the sparsity induced by the soft-thresholding operator. When λ is set to a small value, the algorithm does not sufficiently penalise small values in the attack vector, which leads to higher error in the attack estimation: many non-zero attack components remain undetected. On the other hand, the state estimation tends to be more accurate, as some of the unfiltered attack influence is absorbed into the estimate of x . However, this comes at the cost of robustness, since the algorithm effectively explains the corrupted measurements by adjusting x , instead of correctly identifying the presence of attacks.

Meanwhile, when we increase λ , the algorithm becomes more aggressive in enforcing sparsity. Initially, this leads to a reduction in the attack estimation error, as we can see a faster convergence rate in respect to the other simulation. However, when λ becomes too large, even relevant attack components may be suppressed. This results in a degradation in both attack and state estimation performance. The algorithm forces the solution toward overly sparse vectors, mistakenly eliminating meaningful components of a , and shifting their effect into the state estimate, which in turn increases the error in x .

The variable ν is important in two ways, as a step-size in the gradient step $a(k) - \nu(Cx(k) + a(k) - y)$, and also as a multiplier of λ in the soft-thresholding operator. With these considerations the graph is sure to be influenced by changes in values of ν , we expect that for small values there will be a weaker inertia, and this will make the ratio of convergence smaller increasing stability in turn.

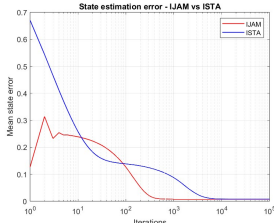


Figure 7: $\nu_{IJAM} = 0.3$

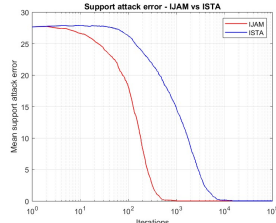
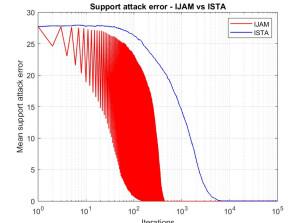
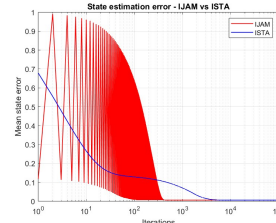


Figure 8: $\nu_{IJAM} = 1$



Now we can try to increase the number of attacked sensors to test the model. The graphs clearly show that as the number of attacks h increases, the algorithm converges to a value different from zero. Even with a single added attacker, ISTA converges to approximately 0.8, whereas IJAM converges to around 0.6. The difference becomes more pronounced with 4 and 5 attacks: ISTA converges to 2.4 and 4, respectively, while IJAM converges to 2.2 and 3.6.

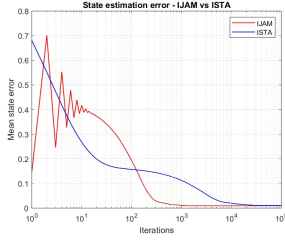


Figure 9: $h = 3$

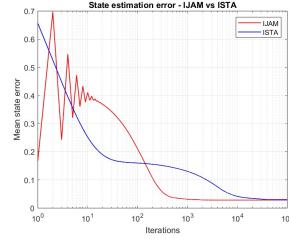


Figure 10: $h = 4$

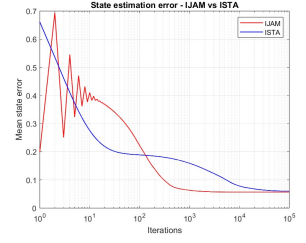
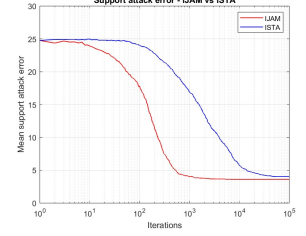
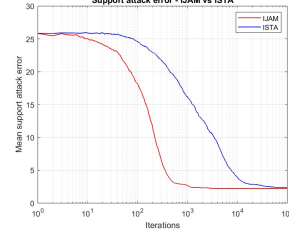
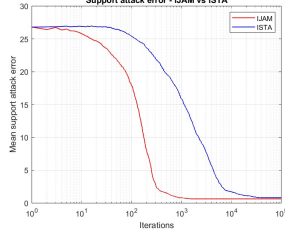


Figure 11: $h = 5$



2 Task 2: Target localization under sparse sensor attacks

2.1 Introduction

In the following task we aim to implement ISTA as an indoor localization algorithm using RSS fingerprinting, assuming that we only have one static target to localize. We do not modify the attack parameters since the training phase is assumed to be safe (i.e. D contains no attacks) and the target's position is already known.

2.2 Performance analysis

The algorithm identifies the target in the state vector at position 37, while the support of the attack vector is given by $\text{supp}(a_{\text{tilde}}) = \{1, 10, 14, 16, 17\}$. Now that we know the ISTA algorithm works, we test whether IJAM gives us different results.

We worked with the matrix $G = \text{normalize}([D, I])$ so that we could operate with the augmented state $z(k) = \begin{bmatrix} x(k) \\ a(k) \end{bmatrix}$, apply our modifications and the soft-thresholding operator to the whole state.

To use the IJAM algorithm, however, we need to be able to freely modify the x -part of the augmented state, setting it as $x(k+1) = C^\dagger(y - a(k))$. To achieve this, we explored two different approaches. In the first one, we split the matrix $G \in \mathbb{R}^{120 \times 20}$ into two submatrices suited for our calculations, combining the state and attack vectors when applying the soft-thresholding operator. In the second approach, we attempted to work directly with the matrix D . Despite these efforts, we observed that in both cases the algorithm doesn't reach the desired solution, even when varying the values of ν and λ .

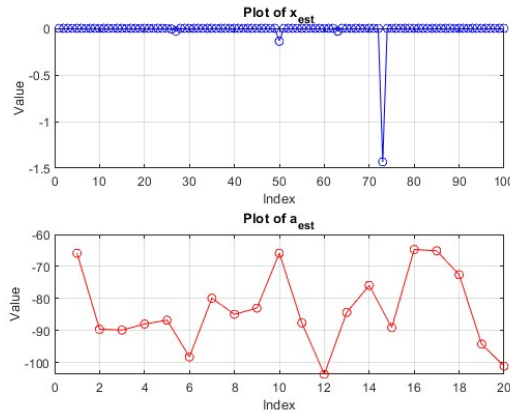


Figure 12: IJAM Algorithm localization

This issue arises because the IJAM algorithm attempts to solve a least squares problem. However, when $q < n$, the problem becomes underdetermined and admits infinitely many solutions. As a result, the Moore–Penrose pseudoinverse cannot be computed, making it impossible to solve the problem using IJAM in this case.

3 Task 3: Secure state estimation of a dynamic CPS with sparse sensor attacks

3.1 Introduction

This task attempts to estimate the sparse sensor attack vector influencing a dynamic cyber-physical system. We want to implement the deadbeat and non-deadbeat sparse soft observer (D-SSO and SSO) to obtain an online tracking that estimates the state and attack vector. Here the classic approach with a Luenberger observer is not a good strategy, because when we have at least one of the eigenvalues of matrix A equal to one, then the system is not observable, even with constant attacks. In D-SSO and SSO we exploit the sparsity of a to avoid this problem, using the soft-thresholding operator seen in task 2.

3.2 Performance analysis

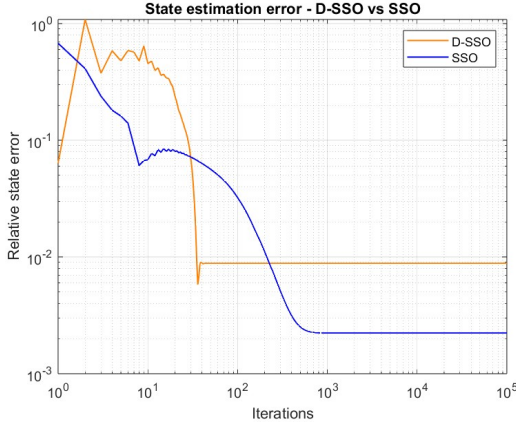


Figure 13: State estimation error

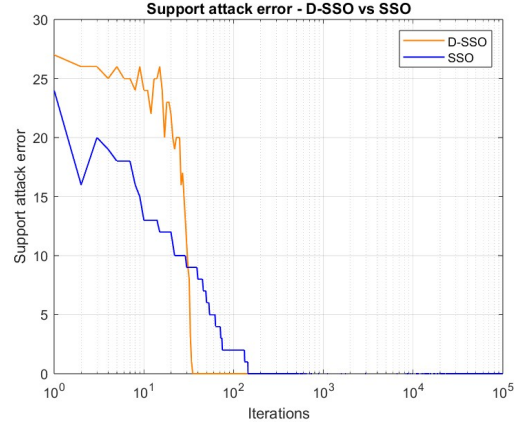


Figure 14: Support attack error

As shown in the graphs, both algorithms converge in finite time. Moreover, their trends closely resemble those of the IJAM and ISTA algorithms, which is expected given that they are online counterparts of these offline methods. We observe that the DSSO algorithm clearly outperforms SSO in terms of convergence rate. After approximately 35 and 50 iterations, the observer successfully identifies both the attack and the state vector, whereas the slower SSO requires around 145 iterations to track the attack support error and up to 10^3 iterations to approximate the state.

The solutions can be further refined by leveraging the fact that, at a certain point in the execution, the algorithm reliably estimates the attack support (in our case around iteration 10^2). Using this information in the state estimation part of the algorithm, we have a solution that ignores the attacked sensors, giving us a system that is essentially attack-free. To implement this, we set to zero the rows of C corresponding to the estimated attacked sensors (in our case sensors 9, 16 and 26). This modification requires updating the observer gain matrix L , since we have changed the structure of the output matrix C .

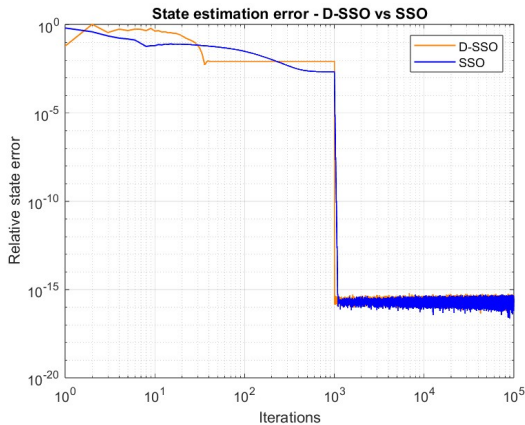


Figure 15: State estimation error

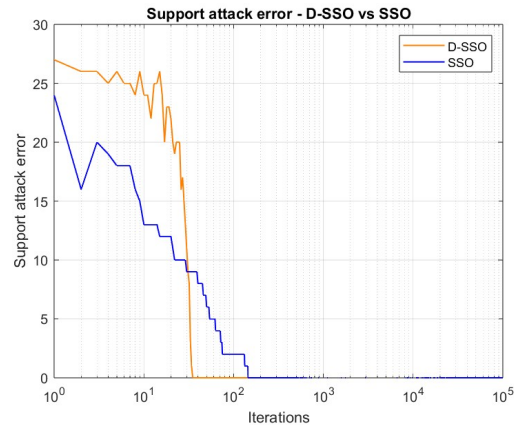


Figure 16: Support attack error

4 Task 4: Target tracking under sparse sensor attacks

4.1 Introduction

Similarly to task 2, this is a RSS fingerprinting problem, with the added challenge of a moving target under a sparse sensor attack.

The system dynamics are defined as:

$$\begin{aligned} x(k+1) &= Ax(k) \\ y(k) &= Dx(k) + \tilde{a} + \eta \end{aligned}$$

4.2 Performance analysis

As a first question, we need to ask ourselves if the attack free system described by the pair (A, D) is observable. This is done by computing the observability matrix and checking whether it has full rank. In this case its rank is 36 which is exactly the number of rows in the A matrix. This confirms that the system is observable in the absence of attacks. Now that we know the attack-free system is observable, we can consider the case in which the system is under attack, starting with constant attacks.

To address this, we consider the augmented state $z(k+1) = \begin{bmatrix} A & 0 \\ 0 & I \end{bmatrix} z(k)$ with dynamics matrix G (where $G = \text{normalize}([D, I])$).

Computing the rank of the observability matrix shows us that it is not full rank (having rank 49 instead of 51), this implies that the augmented system is not observable, so we cannot implement the Luenberger observer strategy as it won't hold in the general case. However, we still need a method to observe the system, so to bypass this issue we make use of the sparsity of both the attack and the state vector, applying the soft-thresholding operator directly to the augmented state $z(k)$.

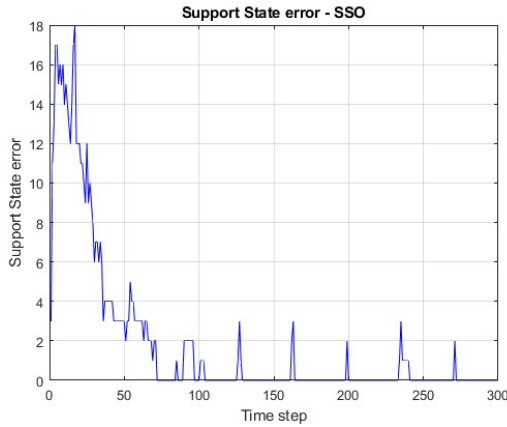


Figure 17: Support state error

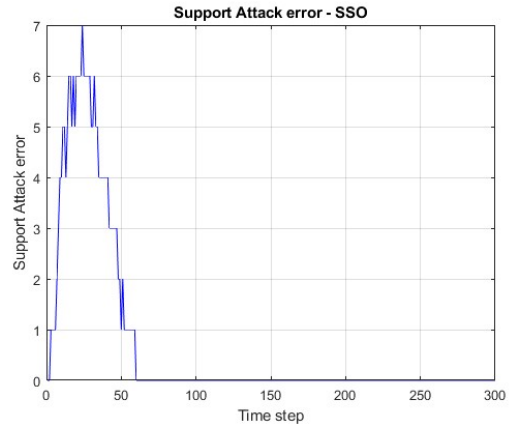


Figure 18: Support attack error

In Figure 17, we observe that the support state error converges to zero within the first 75 iterations. Although there are occasional spikes beyond this point, the error generally remains low, indicating that the algorithm is able to consistently recover the correct state after initial convergence.

In Figure 18 the convergence is faster: the error peaks at around iteration 20 and drops to zero before iteration 60. These results confirm that the SSO algorithm is capable of identifying both the attack and the state support, although at different rates.

In this particular exercise, we might consider whether it is also possible to implement a DSSO version of the algorithm. However, both theoretical analysis and empirical testing show that this is not feasible. DSSO is an extension of the IJAM algorithm to dynamic systems and, as established in Task 2, it is impossible to compute $C^\dagger$ when the condition $q < n$ holds. In this case, the associated least squares problem becomes underdetermined and admits infinitely many solutions, preventing a unique and valid computation of the pseudoinverse.

The failure of the DSSO approach is also evident from the results shown in the following graphs.

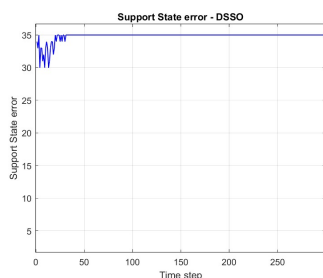


Figure 19: Support state error

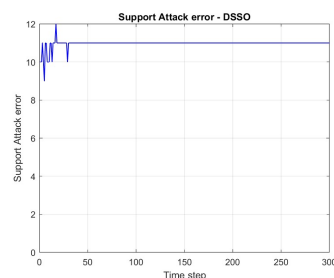


Figure 20: Support attack error

5 Task 5: Distributed target localization under sparse sensor attacks

5.1 Introduction

In this last task, we shift from a centralized version of our algorithms to focus on the distributed form of ISTA (DISTA). In this approach, each node processes its own information locally and shares its estimates only with neighboring nodes. This behavior offers several advantages: privacy, since each node has access only to its neighbors' information; robustness, as the system does not rely on a single fusion center that must remain operational; and scalability, because using a network of relay nodes is far more practical than requiring direct connections to a central node.

We evaluate the performance of DISTA under two network topologies: RING and STAR. For both cases, we assume perfect synchronization among agents and that each of them collaborates properly.

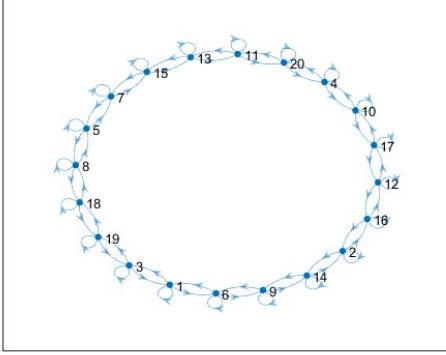


Figure 21: Ring topology

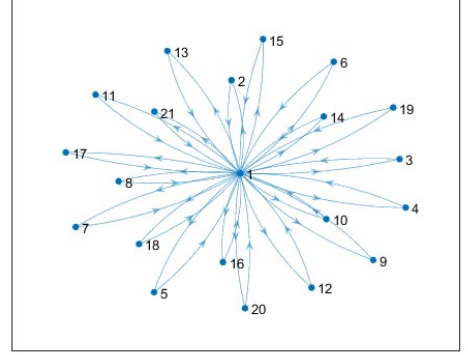


Figure 22: Star Topology

5.2 Performance analysis

Our final results show that we can successfully identify both the position of the target (in cell 37) and the attacked sensors ([1, 10, 14, 16, 17]) using the DISTA algorithm across both topologies. The algorithm achieves high accuracy, and the final solution matches the one obtained via a centralized approach, as seen in Task 2. However, it offers all the advantages of a distributed system: privacy preservation, scalability, and fault tolerance.

The two algorithms differ slightly due to the presence of $z(0)$ in the star topology, which acts as a central node where the average of all other nodes' states is computed. This structure provides better computational performance but is less robust compared to the ring topology. If the central agent z_0 is attacked, the entire star topology would fail. The ring topology is also quite fragile; a better solution than both of these topologies would be a fully connected structure, where each agent communicates with all the others.

Furthermore, the task proposed a refinement step for the attack estimates, setting to zero all components with a magnitude less than 1. Unfortunately, we observed that applying this threshold would eliminate all estimated values, reducing them to zero. Therefore, we adopted a different threshold for each algorithm to preserve meaningful estimates.

The implementation of the IJAM algorithm, as done in Tasks 2 and 4, was not attempted because it requires the exchange of information that has been defined as non-shareable in the distributed setting (specifically, the matrices D and y). A potential workaround could involve introducing a Fusion Center; however, this would contradict the fundamental principle of in-network processing without a central coordinating node.